

摊还分析

二进制计数器递增

- k -bit 二进制计数器: $A[0..k-1]$

$$x = \sum_{i=0}^{k-1} A[i] \cdot 2^i$$

INCREMENT(A)

1. $i \leftarrow 0$
2. **while** $i < \text{length}[A]$ **and** $A[i] = 1$
3. **do** $A[i] \leftarrow 0$ ▷ *reset a bit*
4. $i \leftarrow i + 1$
5. **if** $i < \text{length}[A]$
6. **then** $A[i] \leftarrow 1$ ▷ *set a bit*

k -bit 二进制计数器

Ctr	A[4]	A[3]	A[2]	A[1]	A[0]	
0	0	0	0	0	0	
1	0	0	0	0	1	
2	0	0	0	1	0	
3	0	0	0	1	1	
4	0	0	1	0	0	
5	0	0	1	0	1	
6	0	0	1	1	0	
7	0	0	1	1	1	
8	0	1	0	0	0	
9	0	1	0	0	1	
10	0	1	0	1	0	
11	0	1	0	1	1	

k -bit 二进制计数器

Ctr	A[4]	A[3]	A[2]	A[1]	A[0]	单次 代价	总代 价
0	0	0	0	0	0		0
1	0	0	0	0	1	1	1
2	0	0	0	1	0	2	3
3	0	0	0	1	1	1	4
4	0	0	1	0	0	3	7
5	0	0	1	0	1	1	8
6	0	0	1	1	0	2	10
7	0	0	1	1	1	1	11
8	0	1	0	0	0	4	15
9	0	1	0	0	1	1	16
10	0	1	0	1	0	2	18
11	0	1	0	1	1	1	19

最坏情况分析

考虑 n 次递增，单次递增的最坏情况时间复杂度为 $\Theta(k)$ 。因此， n 次递增的时间复杂度为 $n \cdot \Theta(k) = \Theta(n \cdot k)$ 。

WRONG! 实际上， n 次递增的最坏时间复杂度为 $\Theta(n) \ll \Theta(n \cdot k)$ 。

Let's see why.

Note: 虽然 $O(n \cdot k)$ 也是正确的时间复杂度，但是这个估计过高。

更紧的分析

Ctr	A[4]	A[3]	A[2]	A[1]	A[0]	单次 代价	总代 价
0	0	0	0	0	0		0
1	0	0	0	0	1	1	1
2	0	0	0	1	0	2	3
3	0	0	0	1	1	1	4
4	0	0	1	0	0	3	7
5	0	0	1	0	1	1	8
6	0	0	1	1	0	2	10
7	0	0	1	1	1	1	11
8	0	1	0	0	0	4	15
9	0	1	0	0	1	1	16
10	0	1	0	1	0	2	18
11	0	1	0	1	1	1	19

n 次递增的代价

A[0] 每1次递增翻转 n

A[1] 每2次递增翻转 $n/2$

A[2] 每4次递增翻转 $n/2^2$

A[3] 每8次递增翻转 $n/2^3$

... ..

A[i] 每 2^i 次递增翻转 $n/2^i$

更紧的分析

n 次递增的代价

$$\begin{aligned} &= \sum_{i=1}^{\lfloor \lg n \rfloor} \left\lfloor \frac{n}{2^i} \right\rfloor \\ &< n \sum_{i=1}^{\infty} \frac{1}{2^i} = 2n \\ &= \Theta(n) \end{aligned}$$

因此，单次递增的**平均代价**为 $\Theta(n)/n = \Theta(1)$.

摊还分析(amortized analysis)

摊还分析(*amortized analysis*)是一种分析一个操作序列中所执行的所有操作的平均时间分析方法。很多情况下，单个操作的最坏时间可能很差，但平均时间却不一定。

要注意的是，这里的平均时间和概率没有关系！

- 摊还分析保证了最坏情况下的平均时间复杂度

摊还分析技术

三类摊还分析技术:

- 聚合分析(the *aggregate* method),
- 核算分析(the *accounting* method),
- 势能分析(the *potential* method).

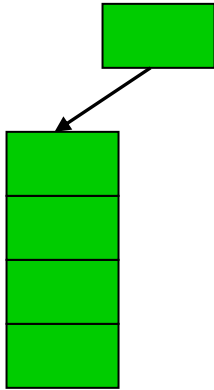
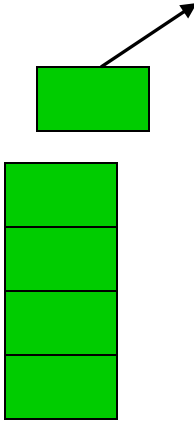
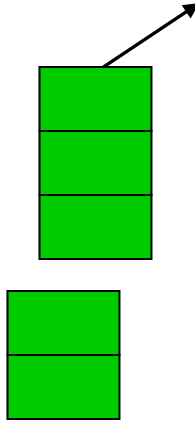
对于二进制计数器的分析，就是聚合分析

聚合分析较为简单，但是相对难以操作。核算分析和势能分析有时会更简单。

聚合分析

- 证明对于一个包含 n 个操作的序列，执行完这个序列的最坏情况时间复杂度为 $T(n)$
- 因此在最坏情况下, 单个操作的平均代价或者摊还代价(*amortized cost*) 为 $T(n)/n$.
- 摊还代价也适用于序列中有多种不同操作的情况

栈操作

三种操作:			
	Push(S,x)	Pop(S)	Multi-pop(S,k)
Worst-case cost:	$O(1)$	$O(1)$	$O(\min(S ,k))$ $= O(n)$

Amortized cost: $O(1)$ per op

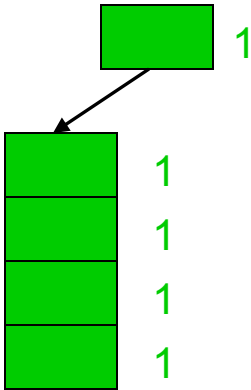
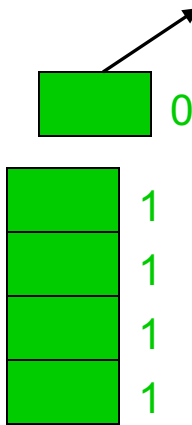
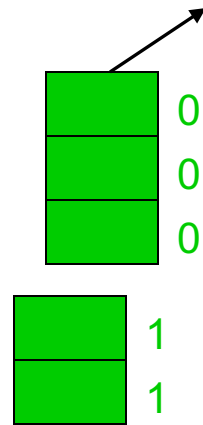
栈操作

- 考虑 n 个push, pop, Multipop操作组成的序列
 - Multipop的最坏情况时间复杂度为 $O(n)$
 - 总共有 n 个操作
 - 因此，序列的最坏情况时间复杂度为 $O(n^2)$
- Observations
 - 每个被压入(push)的对象只会被弹出(pop)一次
 - 假设 n 个操作中涉及 e 个对象的入栈和 d 个对象的出栈，则总的cost是 $2*d+1*(e-d) = e+d$ ，总操作数 $\leq e+d$ ，平均时间复杂度为 $O(1)$
- 注意没有引入概率

核算分析 (Accounting Method)

- 对第 i th 次操作赋予一个虚拟的代价 $\hat{c}_i$ ，称之为**摊还代价(amortized cost)**，可以理解为用\$1支付 1个单位的工作
 - 对于不同的操作，赋予不同的摊还代价(amortized cost)
 - 有些比实际代价高，有些则相反
- 设想有一个银行，其**信用(credit)**为接下来操作所需要的代价
- 银行信用必须为正，我们要求对于所有的 n ，有
$$\sum_{i=1}^n c_i \leq \sum_{i=1}^n \hat{c}_i$$
- 因此，总的摊还代价是总代价的一个上限

栈操作的核算分析

三种操作			
	Push(S,x)	Pop(S)	Multi-pop(S,k)
•赋值代价:	2	0	0
•实际代价:	1	1	$\min(S ,k)$

Push(S,x) pays for possible later pop of x.

Stack Example: Accounting Methods

- 压入一个对象摊还代价为\$2
 - \$1 给压入操作
 - \$1 预备给可能的pop或者multipop操作
- 每个栈内的对象有预备的\$1, 因此银行的信用永远不可能为负
- 因此, 总的摊还代价= $O(n)$ 是总代价的一个上限

二进制计数器递增的核算分析

对于每次一个bit由0转为1，我们缴纳\$2

- \$1 用于实际的转化.
- \$1 预存用于下次将 1 转为 0.

在任意时刻，每个1 bit总有\$1 信用，用于重置1到0.
(重置1到0是“**免费**”的)，因此总有信用非负， $\sum_{i=1}^n c_i \leq \sum_{i=1}^n \hat{c}_i$

Example:

0 0 0 1\$1 0 1\$1 0

0 0 0 1\$1 0 1\$1 1\$1

0 0 0 1\$1 1\$1 0 0

Cost = \$2

Cost = \$2

二进制计数器递增的核算分析

INCREMENT(A)

```
1.  $i \leftarrow 0$ 
2. while  $i < \text{length}[A]$  and  $A[i] = 1$ 
3.     do  $A[i] \leftarrow 0$        $\triangleright$  reset a bit
4.      $i \leftarrow i + 1$ 
5. if  $i < \text{length}[A]$ 
6.     then  $A[i] \leftarrow 1$      $\triangleright$  set a bit
```

- 当递增时,
 - line 3的摊还代价 = \$0
 - line 6的摊还代价 = \$2
- 单次INCREMENT(A) 的摊还代价 = \$2
- n 次 INCREMENT(A) 的摊还代价 = $\$2n = O(n)$

势能方法

思想: 将银行账号看为**势能**，将势能释放就可用来支付未来操作的代价

基本思路:

- 假设初始数据结构为 D_0 .
- 第 i 次操作将 D_{i-1} 转化为 D_i .
- 第 i 次操作的代价为 c_i .
- 定义一个**势函数**(*potential function*) Φ :
 $\{D_i\} \rightarrow \mathbb{R}$,
使得 $\Phi(D_0) = 0$ 且 $\Phi(D_i) \geq 0$, 对所有的 i .
- 基于 Φ 摊还代价 **amortized cost** $\hat{c}_i$ 被定义为 $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$.

势能分析

- 与核算分析类似，但是将信用看成数据结构的势能
 - 核算分析单独存储每个对象的信用，势能分析考虑整个数据结构的势能。
 - 当前的势能可以用于支付未来操作的代价
- 势能分析是三种摊还分析中最为灵活的

对势能的理解

$$\hat{c}_i = c_i + \underbrace{\Phi(D_i) - \Phi(D_{i-1})}_{\text{势能差 } \Delta\Phi_i}$$

- 如果 $\Delta\Phi_i > 0$, 则 $\hat{c}_i > c_i$. 第 i 次操作增加势能, 为未来操作提供能量
- 如果 $\Delta\Phi_i < 0$, 则 $\hat{c}_i < c_i$. 数据结构降低势能来支付该次操作的费用

总摊还代价与总真实代价的关系

n 次操作的总摊还代价为

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n (c_i + \Phi(D_i) - \Phi(D_{i-1}))$$

两边相加

总摊还代价与总真实代价的关系

n 次操作的总摊还代价为

$$\begin{aligned}\sum_{i=1}^n \hat{c}_i &= \sum_{i=1}^n (c_i + \Phi(D_i) - \Phi(D_{i-1})) \\ &= \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0)\end{aligned}$$

两者相差最终势能-初始势能

总摊还代价与总真实代价的关系

n 次操作的总摊还代价为

$$\begin{aligned}\sum_{i=1}^n \hat{c}_i &= \sum_{i=1}^n (c_i + \Phi(D_i) - \Phi(D_{i-1})) \\ &= \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) \\ &\geq \sum_{i=1}^n c_i \quad \text{注意 } \Phi(D_n) \geq 0 \text{ 且 } \Phi(D_0) = 0.\end{aligned}$$

栈操作的势能分析

Define: $\phi(D_i)$ = 栈中对象的个数 我们有 $\phi(D_0)=0$.

对于每个操作:

Push: $\hat{c}_i = c_i + \phi(D_i) - \phi(D_{i-1})$
 $= 1 + j - (j-1)$
 $= 2$

因此, 单个操作的摊还代价为 **$O(1)$**

Pop: $\hat{c}_i = c_i + \phi(D_i) - \phi(D_{i-1})$
 $= 1 + (j-1) - j$
 $= 0$

Multi-pop: $\hat{c}_i = c_i + \phi(D_i) - \phi(D_{i-1})$
 $= k' + (j-k') - j$
 $= 0$

$k' = \min(|S|, k)$

INCREMENT的势能分析

定义第*i*次操作后，计数器的势能 $\Phi(D_i) = b_i$ ，
即计数器中1的个数

Note:

- $\Phi(D_0) = 0$,
- $\Phi(D_i) \geq 0$ for all i .

Example:

0 0 0 1 0 1 0

$\Phi = 2$

(0 0 0 1\$1 0 1\$1 0

核算分析

计算摊还代价

假设第 i 次 INCREMENT 重置了 t_i bits (line 3).

实际代价 $c_i = (t_i + 1)$

这次操作结束后1的个数: $b_i = b_{i-1} - t_i + 1$

因此, 第 i th次 INCREMENT 的摊还代价为

$$\begin{aligned}\hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= (t_i + 1) + b_i - b_{i-1} = (t_i + 1) + (b_{i-1} - t_i + 1) - b_{i-1} \\ &= (t_i + 1) + (1 - t_i) = 2\end{aligned}$$

因此, n 次 INCREMENTS 的最坏时间复杂度为 $\Theta(n)$